

How Accenture Industrializes Machine Learning Workloads on AWS

October 2020



Notices

Customers are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

© 2020 Amazon Web Services, Inc. or its affiliates. All rights reserved.



Contents

Introduction	1
The Evolution of Machine Learning (ML) Engineering – Bridging the Gap.....	1
The AI/ML Product Lifecycle	3
Addressing the Needs.....	3
Understanding the Challenges	4
How Accenture builds an ML Engineering Foundation on AWS	9
Conclusion	26
Call to action	26
Contributors	27
Document Revisions.....	27

Abstract

As companies start to leverage artificial intelligence (AI) for their business, research shows that they must focus on scaling AI to get the necessary competitive edge¹. A critical aspect of scaling AI involves focusing on how to scale ML as a service to business. This whitepaper outlines a framework and tooling Accenture built on Amazon Web Services (AWS) that enables operationalization of ML models, from a data scientists' desktop to production, in an industrialized, automated, and repeatable way.

Introduction

In this digital age, Intelligence is now synonymous with big data and artificial intelligence (AI). Over several years now, we have seen the trends around re-engineering of our data supply chains, data lakes and journeys to cloud in an effort to round up data at scale and drive intelligent outcomes. The rich pile of data, collected from business systems over the past years, have naturally positioned us to execute very large experimentations for solving business problems at hand. The recent advancements in and availability of several AI/machine learning (ML) technologies have added fuel to it.

Companies, who have acted quickly, are realizing the benefits of harnessing and having access to large digital datasets. The stage is set for industries to deploy widespread business applications, that are learning and insights driven, delivering differentiated outcomes. Broadly, however, realized value from AI/ML remains significantly below what was originally perceived. Sustained outcomes, reliably driving everyday business processes, are fewer and far apart. In the experience of a US automotive giant, “Even with a team of 250+ data scientists, struggle continues with data science operationalization. The challenge is shifting from the ability to write good AI/ML algorithms to having them deployed and operating successfully for business functions producing sustainable outcomes”.

The Evolution of Machine Learning (ML) Engineering – Bridging the Gap

Machine Learning Engineering is the use of scientific principles, tools, and techniques of machine learning and traditional software engineering to design and build complex computing systems². ML Engineering brings the rigor that enables AI algorithms to be operationalized at scale enabling the enterprise to run intelligent applications in a business-critical environment. It complements what is left out of data science and data engineering that makes a ML lifecycle complete.

The Gap Between Experimentation and Operationalization

This decade saw a rapid evolution of big data and AI/ML technologies³. Rich pile of data collected over the past years have naturally positioned companies to leverage AI/ML to deliver business impacts, competitive edge and innovative products. While a lot of companies have invested significantly in AI/ML experimentation and data science, a lot is left in the area of AI/ML operationalization under delivering on the promise. The difference in return on AI investments between companies in the proof of concept stage

and strategic scalars is significant⁴. Those who have invested are truly reaping the benefits of the differentiating outcomes that were perceived.

So, what is ML Engineering and how is it different from the topics of data science, data engineering or ETL Engineering and why is it important and relevant now? Data science have been about building and experimenting with models in an exploratory environment. Data scientists focus on working with multiple⁵ models, testing, tuning and evaluating them for getting to the best possible outcome for relevant use cases.

Data Engineering has traditionally been about enabling data science with correctly engineered data and the scale to be able to process it effectively. ETL engineering has been limited to primarily processing relational data through available tools in the market. It is evident that although individually these fields add a lot of value, the collective impact of ML is not fully realized until an industrialized outcome is handed over to business operations. The question is still about the impact these silos of data science and data engineering activities have created. The necessity to bridge this gap has given rise to the field of ML Engineering.

The Trend

ML Engineering begins by facilitating the experimentation phase of data science but continues with deployment, run, monitoring and feedback of the ML products embedded into business applications. It's a self-fulfilling industrialized pipeline of components orchestrated to continuously monitor and perform against a targeted benchmark of desired metrics.

Done correctly, this hardens the baseline models coming out of data science experiments for production; manages multiple deployment patterns; enables governance and tracking during run; monitors target performance taking corrective actions on decays; providing transparency to the results at the same time.

Data science used to focus on building and training models and algorithms. Data Engineering streamlines data acquisition, curation and management pipelines. ML Engineering complemented these two fields of practice, bridging the gaps, and made AI/ML complete.

However, ML Engineering is not without its own set of challenges. A strong set of standards, rigor and governance is required to practice it effectively and at enterprise scale. In this whitepaper, we discuss the challenges for building up those standards and practices. We also introduce an architectural framework of components for industrializing the approach to ML Engineering.

The AI/ML Product Lifecycle

Addressing the Needs

Most companies have been engaged in model development and training activities for some time now. As technologies evolved from more traditional standalone statistical models into widespread development of ML and DL models, a lot of business opportunities opened up. As businesses tried to get the most out of data science outcomes, a lot of challenges started surfacing at the same time. The challenges started with the proliferation of available alternatives in programming languages, technical libraries, infrastructures solutions, and with the shift towards large scale distributed models for digital data.

As the data science lifecycle became more matured with these evolutions, it stretched deeper into realms beyond monolithic experimentation, and beyond a singular approach of productization. Newer asks emerged around how AI should be used, sensibly and with no bias, for augmenting decision making. Managing transparency around responsibility, explainability became important topics for Applied Intelligence. Then, came the question of how to sustain the outcomes from these learning models over time. Could they continue to deliver meaningful business impacts on a regular basis without losing their efficiency and impact?

Throughout this paper, we briefly discuss the phases of an AI/ML product lifecycle as shown in the figure below. This figure shows the cycle from data to an operationalized ML application product, delivering sustained outcomes. The cycle starts with the identification of a business problem and then associating a ML enabled software product or solution to address the business problem.

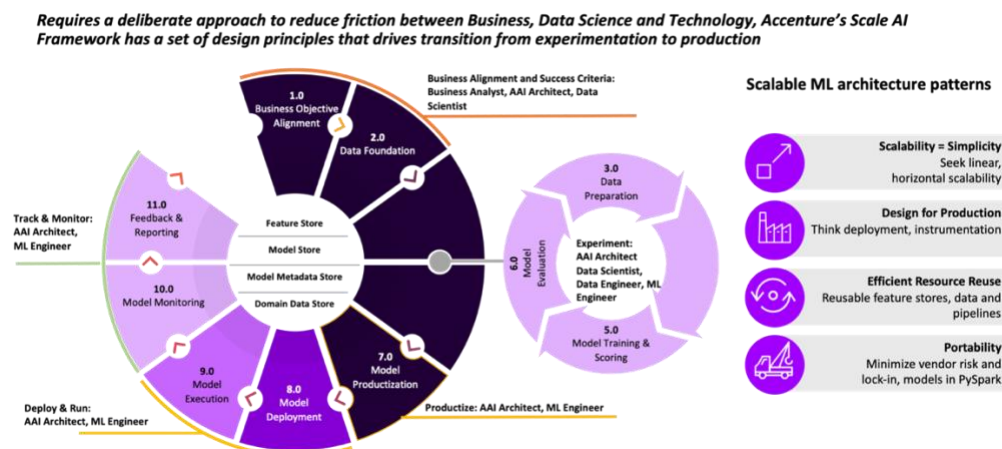


Figure 1 – Accenture's High-Level ML Lifecycle

The experimentation phase develops a set of learning models that can be applied to solve the problem. Domain datasets are made available in an exploratory environment for ML experimentation. During this phase an extensive set of features are created and managed for experimenting with the models. Models are developed, trained and evaluated for deployment readiness. Complete model data is stored, training runs are tracked, and testing insights are used for evaluation. Existing decayed models are also re-calibrated similarly. Approved models are containerized and deployed to edge locations. Feature stores either merged with domain data or pipelined in containers for use in a business-critical operational environment. Deployed models are calibrated, [A/B tested](#) and put to use. Local cache of model outcomes delivers results through API interfaces for applications front-ending a user. During operations, models are closely monitored for performance and decay then calibrating actions are taken as appropriate. Model feedbacks are pipelined back into experimentation for re-tuning and calibration.

Understanding the Challenges

The Drivers for a Robust Architectural Foundation Supporting the Lifecycle

Many challenges continue to plague this lifecycle and inhibit businesses from seamlessly operating ML enabled applications at scale. These challenges in addition to the inherent needs of an AI solution, drives the considerations for a good foundation. Key questions must be asked and addressed throughout the process.

Long Cycles of Feature Engineering

In this critical phase of experimentation, the cycle time and effort to identify the *right* features continue to be huge. Companies need to be more efficient with finding and leveraging relevant pre-defined features or correlating features with data versions. Thinking about how your model calibration efforts get impacted by difficulties with correlating the models in operation along with associated features or data versions. When thinking about how it looks to accelerate, consider the following questions:

- How much time are your data scientists spending to find the right data sets, data versions and features for their models?
- Are they effectively able to collaborate across company silos?
- Do they have right tools and automation in place for speed to develop the right features and reduce dimensionality?

Low productivity of Model Development & Training

It is important to have the right data, platform and infrastructure in place for your company. Complexities in handling multiple languages, such as R, or Python also need careful consideration, consider the following questions:

- Are they impacting critical activities like AB testing and simulation testing?
- Do you have rights skills to build different types of models for optimal outcomes?
- Is your data science team able to get the right visibility into shapes, characteristics, and distribution of data quickly?
- Do you have an optimal infrastructure and data science platform?

Your data scientist can have a great model coded in PySpark, but if it can't successfully run or takes too long, you run the risk of derailing your experimentation effort. A major retailer's data science team struggled for a long time with a few hundred Kbytes of data on a sub-optimal Spark platform that was expected to scale to terabytes of data.

In an industrialized environment, having multiple optimal configurations and infrastructure for different workloads is critical. Consider the following questions, when building your infrastructure:

- Do you have guidelines for deployment and execution of different ML patterns?
- Are your infrastructure resource allocation techniques slowing you down?
- Can your data scientists' self-service infrastructure scale dynamically for experimentation and model development?

Model Evaluation and Calibration Not Engineered to Scale

Model evaluation is a key step to deploying the right set of models or creating an ensemble from a very large set of alternatives in testing. When evaluating models, you should consider the following questions:

- Do you have a consistent approach, within a given business problem around the right set of decision metrics (e.g. Confusion Matrix), for making the right selections?
- Do you have the right roles and governance in place for contributors and decision makers?
- Do they have access to required information to orchestrate a decision like this?

- Have you standardized on what the governance checkpoints are and who the players are?

These questions apply to new models, and to the re-calibration of models already deployed in operations.

Governance is backed by data. You should also consider the following questions:

- Are you capturing enough metadata during the lifecycle to be able to govern experimentations and make decisions? Have you thought about having this as a logical extension of your metadata captured during data engineering?
- Are you tracking and documenting model experiments and associated data (e.g. features, hyper-parameters, tunings, accuracy, performance), do you have a versioning mechanism in place?
- How easy is it to discover and share your models?

Low maturity in Model Deployment standards

ML algorithms, platforms, and infrastructure continue to evolve, as a result establishing a good deployment architecture must address multiple patterns. For example, algorithms that are GPU based or implemented on a single node might require docker framework; whereas Spark based models might need an efficient way to deploy in a cluster. There are many variations and combinations for an optimal deployment, consider the following questions:

- Do you have your standards in place to cover all the model variations?
- Are your containers able to support models written in different languages? We have seen companies invest in building their own platform. But that may not be feasible for many. What is your approach?

Questions about governance questions can also extend in to the deployment space. Mature companies have thought through the following:

- At what point a model is ready to be deployed and what is the deployment checklist to be signed off?
- How often should you deploy your model to production and on what basis?
- What artifacts are required to be submitted for a deployment approval?
- Which roles should be part of the approval workflow?
- Have you collected all the meta-data required by this process?

Lack of alignment between Data Science and Data/ML Engineering

The current practices of data science and ML engineering remain divergent. Often these two teams have different coding practices and standards that leads to serious challenges during deployment. Do you face crucial contention around the responsibilities of translating the *baseline* models to *secondary* models? The experimentation models are built using python, R, or other data scientist friendly tools and platforms. This may not be the optimal way to deploy in a large-scale operational environment.

Having silos of data science and engineering, that don't make the ML pipelines a natural extension of data engineering pipelines, is a critical gap. Consider the following questions:

- What is your approach in ensuring consistency of model performance during this transition?
- Have you established the right governance and trackability of consistent performance metrics between primary and secondary models? If not, you get seriously impacted by model performance in operations and feasibility of deploying those models.
- Can deployed models be parameterized for tuning attributes dynamically?
- What kind of ownership should be distributed between engineers and scientists'?
- How would you decouple a model from the application that leverages it?

Difficulties in Tracking Model Drift and Decay

Frameworks and standards are at their early stages of maturity for tracking a model performance in operations using appropriate information from a feature store, metadata store, and model repository and variance. Consider the following questions:

- How to track if model is running optimally and what is right threshold for accuracy/precision?
- How and who decides on model drifts and what action should be taken, e.g. re-structure, recode etc.?
- Who decides that model is nearing *end of life*?
- How often do you re-train and re-deploy the models?

Bias and Explainable AI

The effectiveness of AI systems can be severely limited by the machine's current inability to explain their decisions and actions to human users. This poses risk to adoption and trust of these solutions. In a recent incident, an online retail major's recruiting application had to be discarded because of the inherent gender bias. Questions have been raised about crime detection systems that demonstrate a racial bias developed out of the data leveraged in ML lifecycle.

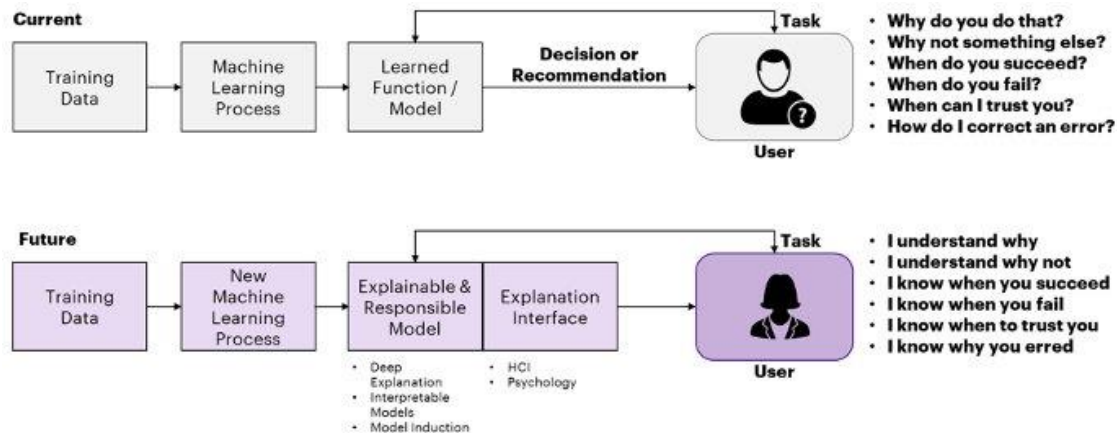


Figure 2 - Challenges with Bias and Explainable AI

“AI is sending people to jail – and getting it wrong” was written in a MIT Tech Review⁶. Explainable AI is essential for business users to understand, trust, and effectively manage the decisions emerging out of deployed ML applications. Regulations such as GDPR have already put constraints on automated decision making. More specific regulations to manage AI/ML driven decisions are expected in the future.

AI/ML black box algorithms offer little insight into how it reaches its outcomes, because many algorithms are beyond human comprehension. Essentially, the challenge is to identify and expose a set of metrics that provides enough justifications to the path to an outcome. This is shown in Figure 2. In addition to creating a suite of techniques that produce explainable models, at the same time, learning and execution performance can't be compromised.

Managing bias and interpretability are the two most essential components that deliver transparency for successfully deploying business applications.

Lacking Standards for an Ethical and Responsible AI

ML applications learn from data, so that data must be representative of the real world. Consider the following questions:

- Do these learnings respect the privacy of individuals?
- Do the learnings demonstrate any inherent bias to certain individuals or groups?

To ensure AI models are fair, companies should be able to identify data or algorithmic biases to understand what influences a model's decision. A key question and consideration should include the following question: Is the AI appropriately governed?

Just as businesses audit financial and operating practices, they must also be able to audit their AI systems. AI is currently developed and maintained in silos, creating confusion and uncertainty around inspections and assessments. Enterprise-wide accountability is required. Consider the following question: Is the AI legal, ethical, and moral?

Companies must use AI in a way that aligns with their mission and social responsibilities, be able to evaluate the technology's impact on employees and customers, and operate within the boundaries of regulation, both enacted and in discussion.

How Accenture builds an ML Engineering Foundation on AWS

Accenture brings a framework that integrates the key capabilities needed for enterprise scale build, deployment and management of Data Services and ML applications. The framework addresses the following questions:

- How to transition from experiments to production, enabled through an intelligent data supply chain and continuous engineering of an ML lifecycle?
- What are the architectural capabilities that you need to put in place to address these? How many of these are available as 3rd party products?
- How much integration work do you need to take on for a seamless end to end experience?
- How much do you have to build yourself?

In the following sections we provide a detailed look into these questions.

Smart Data Foundation

This is probably the most fundamental capability to get ML engineering heading in the right direction. A robust data foundation will not only ensure that data is quickly accessible for experimentation, it also ensures proper governance is in place for that



data to be trusted and useful. A smart data foundation provides capabilities such as governed onboarding of data, scaled data engineering pipelines, tiered data consumption layers, a data exploration zone, and centralized metadata tracking. It's critical for this foundation to enable data scientists to onboard their own data for experimentation but at the same time provide well defined governance and management of that data. It is important to have an integrated business data catalog for searching, shopping, and provisioning data as self-service.

A Collaborative Framework with AWS for Data Foundation

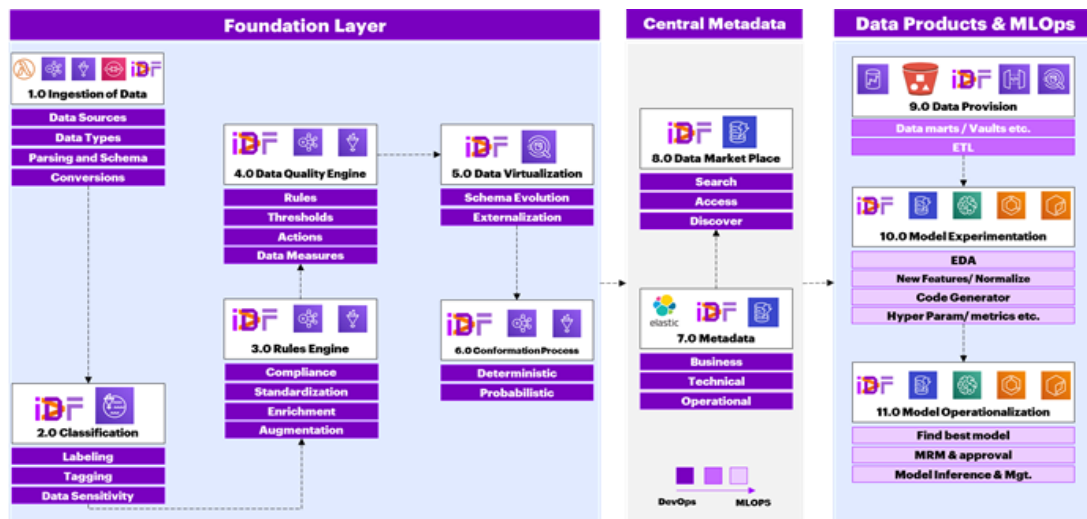


Figure 3 – Intelligent Data Foundation for ML

One of the strengths of the Accenture and AWS collaboration is the enterprise relevance and innovation we bring together in design thinking, architecture layout and solutioning. Intelligent Data Foundation (IDF) is one example of how Accenture and AWS have collaborated to bring in an asset that automates, accelerates, and solves the data foundation layer through use of serverless, AWS specific, and ready to use modules that provides our customers a jump start for building and maintaining data, and machine learning based platforms.

AWS believes that a strong data foundation built on top of governance oriented, metadata centric, and compliance-based strategy ensures a robust data platform that provides the right kind of enablement of artificial intelligence and machine learning platforms. In many of our customer stories we have seen the struggles of data accessibility, data trustiness, and automation being the biggest bottleneck in operationalization of data science.

The above figure describes how IDF as a framework enables the entire data and ML lifecycle. We start by providing a governance first data acquisition component that

enables us to securely bring in data into the data platform through a regulated and registered process. We use a collection of custom and AWS services to build the governance processes that includes a custom UI portal hosted through Amazon CloudFront and workflow management system through collection of AWS Step Functions, Amazon Simple Queue Service (Amazon SQS), and Amazon Simple Notification Service (Amazon SNS) along with custom implementation designed collaboratively.

The central metadata engine ensures that all data that comes into the system is labeled, tagged, classified, profiled, and stored. The metadata layer is built on PySpark that uses AWS services such as AWS Glue, Amazon EMR, Amazon Comprehend, Amazon Rekognition, and Amazon Textract.

The metadata layer is fundamental to all of our contextual solutions around building data and ML products for our customers. As part of the IDF solution, we augment the technical metadata captured by AWS Glue Data Catalog with business and operational metadata and build a centralized metadata layer backed by Amazon Elasticsearch. IDF leverages AWS machine learning services and custom scripts to capture this additional business and operational metadata

The Rules and Data Quality Engine was written ground up and is patented that provides *low-code* and automated rules-based approach for data standardization, augmentation, enrichment, and compliance. The engines run on EMR and Glue and interacts with various components and services like Amazon Simple Storage Service (Amazon S3), Amazon DynamoDB, and AWS Step Functions.

The data virtualization layer leverages Amazon Athena and Amazon Redshift Spectrum that enables a more SQL centric data interpretation angle to the data platform. The data virtualization process is coupled with security and access control using lake-formation through the metadata layer. Our custom component that sits on top of lake-formation using AWS Lambda registers a dynamic role based and attribute-based access policy to different personas who access the data.

The data conformation and harmonization processes are built on top of Glue [FindMatchers](#) and custom solutions that leverage EMR. This enables the data to have trust worthiness and acceptance as the data is getting ready for the business end.

Once the core data foundation is established, the by-product of having a metadata centric and governed data platform is the data accessibility the platform offers by means of enabling search and shop capability. IDF uses AWS services such as AWS Glue Data Catalog and Amazon Elasticsearch apart from enabling 3rd party solutions to build

a data marketplace for data scientists' and ETL engineers to start using the data and build data and ML products.

The *traditional* approach of building and sustaining ETL pipelines are built on templates through vertical knowledge brought by Accenture SMEs on different domains and a wide array of customer success stories and implementations. The ETL solution stack is built using AWS services. This not only provides a great start to our customers on domain-based solutions, but also gains traction through production ready code base and templates.

Additionally, the IDF code generator components are built as Amazon SageMaker and Jupyter Notebook extensions to provide end to end ML operationalization that includes utilities, containerization, model train-test-validation, hyper-parameter selection model deployment, model monitoring, retraining, explainability, and governance.

Integration with Native Capabilities for a Cost Optimized Approach

One of the important features of IDF as a foundational layer is how we have enabled all our customers to design the entire data-platform in a cost optimized way. All the layers around governed data-acquisition, rules-engine, metadata-engine, data-quality engine, and compliance engine are designed around serverless AWS services that enable auto scaling, easy management, less overhead, and easy to automate. Similarly, for the MLOps piece, we also used shared GPUs, spot instances and auto-scaling to help our customers get a highly automated and cost-effective solution around Data and ML Ops.

The following sections focus on the IDF AI ML capability that leverages Amazon SageMaker and operates on an AWS stack (mostly serverless) to provide an AI ML solution for our customers.

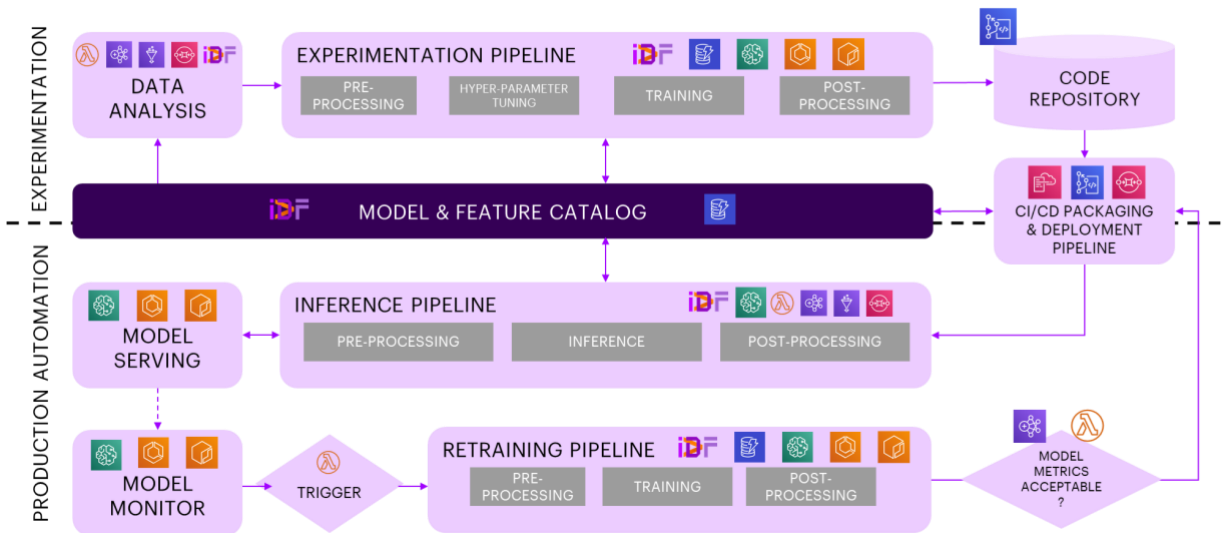


Figure 4 – IDF AI ML solution

The previous figure provides insight into how we have designed and built a robust and comprehensive ML Ops process around Sagemaker. The overview of the solution we offer is the modularized approach and coupled with API's and common libraries that enable us to use a specific set of capabilities and build towards a final solution. Customers who want to start only with the experimentation phase initially can only enable the experimentation modules that enables a collaborative way of designing and implementing models. However, at a later time, if the customer's intention is to extend the capability to production and operationalize models then they can turn on the capabilities without additional effort. The idea is to start small, ensure get the process right and scale big to an all in one approach of enabling components that makes sense for the use case and approach at hand.

Experimentation Platform

An experimentation platform provides key capabilities for data exploration and iterative building, testing, tuning, and evaluation of ML models for deployment. IDF provides several Jupyter extensions that augments Amazon SageMaker capabilities with the goal of providing the following features:

- Templatized code to accelerate ML experimentation
- Standardized and Reproducible ML Pipelines to accelerate ML deployment
- Consistency of code between training and inference to reduce training/inference skew

Feature discovery and Profiling

Once data is accessible and available to the data scientist, one of the first step to start with model creation is to understand the shape, distribution, progression, and overall landscape of data. A lot of the details around data is already gathered through the IDF metadata repository that enable easy identification of profiles, catalogs, glossary, tags, labels etc. However, as part of feature discovery and importance, IDF provides SageMaker custom extensions which are predefined templates of questions and analysis (based on lessons learned during multiple customer delivery) that gives a detailed analysis of the data and how it can be used. These IDF capabilities referenced can be indications of the data quality, data veracity, features within the data set in terms of generating new attributes, and understanding data normalcy. These details not only help the data scientist to understand the data better but also enables them to easily use the IDF SageMaker code generator extension for accelerated model development, training and validation.

The screenshot shows a Jupyter Notebook environment with the 'Exploratory Analysis UI' extension. The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help) and a toolbar with icons for file operations, running, and saving. The main content area displays the 'EXPLORATORY ANALYSIS UI' with the following inputs:

- Enter Table Name:** banking_db.bank_loan_bucket_table
- Enter Target Column Name:** response_binary
- File Type:** GZ

Below the inputs is an orange button labeled 'Q OVERALL ANALYSIS - RECOMMENDATIONS'. The results are displayed in a table:

Recommended Numerical Columns	duration
Recommended Categorical Nominal Columns	job,marital,education,default,housing,loan,month,poutcome,ponse
Recommended Categorical Ordinal Columns	age,balance,campaign,pdays,previous
How many empty rows of data in the dataset	0
Identify for each column the percent distribution of empty/null values	Check 'ALL COLUMNS - ROWS WITH NULLS' Section below
From the string columns identify which are categorical vs non-categorical	Check 'IDENTIFY COLUMNS AS CATEGORICAL OR STRING' Section below
Overall Skewness of the Target Variable	2.6496950828697385
Overall Kurtosis of the Target Variable	5.021135092715414
Distinct values in Target Variable	response_binary_ENCODED response_binary_ENCODED 0 0 35962 1 1 4037 Name: response_binary_ENCODED, dtype: int64
Does the dataset follow normal/Gaussian Curve for Target Variable	NO with stat = 0.3440019488334656 , p = 0.0

Figure 5 – Exploratory Analysis Extension

Experiments

Once the data scientist has the knowledge of the data, he can start the actual work of model generation, testing, training, validating, and tuning, iteratively until they find the *right* solution that they think can be operationalized. The code generator framework makes this entire process of *experimentation* simpler and more effective for the following reasons:

Typically, the entire process of creating models, trying that with multiple objective metrics, data-preprocessing, data-normalization, tuning and finding the best algorithm requires a lot of *boiler-plate* code. The Sagemaker code generator extension automatically *stitches* together all the required steps of a ML pipeline such as pre-processing, training, hyperparameter tuning etc. and helps accelerate both the *experimentation* and deployment process. The process is completely transparent and the data scientist can dive into the details of how the code was created and refine it as required.

Based on prior experience, enterprise data scientist who have been working in the field for many years are driven more by the model result and alignment to the business problem in hand and less excited by having to repeat the same code over and over again. The IDF SageMaker code generator extension helps enterprise data scientists to avoid writing the same code the *over and over* but still enables access to the code for testing, and use their creativity to solve the current problem.

There has always been a challenge in baseline code and secondary code that is generated by a data scientist versus the code that goes to production. Standardizing the code not only enables you to avoid a complete re-write of code but makes the *path to production* fast, effective, and agile. Think about enterprises that have to manage and build hundreds of models, this becomes difficult unless the model code is standardized and re-usable.

AWS believes in governance and metadata at every stage. For our customers every experiment needs to be recorded and tracked. Code generator enables us to have inbuilt integrations with feature catalogs and API's to effectively manage and track each experiment.

This is AI ML Framework for Code Generation. Use this screen to automate End to end Models lifecycle through a set of processes. By filling in the form, automate following:

Step 1 - Preprocessing across multiple Sagemaker (includes Standardization, Normalization , Scaling, One-Hot-Encoding etc.).

Step 2 - Launch and Execute Models through train/tested/validation along with Hyperparameter tuning and best model selection.

Step 3 - Inference on unseen data before finalizing model for production.

Step 4 - Automatic packaging of code, artifacts etc. (MRM)

Utilities - EDA

DATA PRE-PROCESSING	MODEL TRAIN/TEST	DATA INFERENCE	PACKAGING/APPROVAL
Model Artifacts Base Location	s3://idf-di-playground-us1/model-base		
Project Name	XGBOOST-SPARK-PROJECT		
AWS Region	us-east-1 us-east-2		
Data Location in Glue DB.Table	playground.employee_attrition		
Container Count for Pre-Processing	2		
Container Type for Pre-Processing	ml.t3.medium		
Classifier/Predictor Type	multi softmax		
Target Variable	attrition		
Columns to be Deleted	employeecount,employeenumber,over18		
Numerical Columns	age,dailyrage,distancefromhome,education,environmentalsatisfaction,hourlyrate,jobinvolvement,joblevel,jobse		

Figure 6 – Code Generator Extension

The above figure provides an insight into the Code Generator extension and how we use Amazon SageMaker containerization and perform the pre-processing using AWS managed analytics and machine learning services. IDF uses a combination of AWS Glue and Sagemaker containers that can help in end to end ML Ops without the need to manage and own clusters for data preparation, pre-processing, or actual model implementation.

This is AI ML Framework for Code Generation. Use this screen to automate End to end Models lifecycle through a set of processes. By filling in the form, automate following:

Step 1 - Preprocessing across multiple Sagemaker (includes Standardization, Normalization , Scaling, One-Hot-Encoding etc.).

Step 2 - Launch and Execute Models through train/tested/validation along with Hyperparameter tuning and best model selection.

Step 3 - Inference on unseen data before finalizing model for production.

Step 4 - Automatic packaging of code, artifacts etc. (MRM)

Utilities - EDA

DATA PRE-PROCESSING	MODEL TRAIN/TEST	DATA INFERENCE	PACKAGING/APPROVAL
Instance Count for Model Build	2		
Instance Type for Model Build	ml.m4.xlarge		
Training Batch Size	1000		
Train/Test Split Ratio (OR)	.1		
Initial K Values	8		
Disk size (in GB) for container	20		
Metric for Hyperparameter Tuning	miogloss		

Figure 7 – Code Generator Parameters

The above figure provides insight on how data scientists can quickly generate code by using and providing parameters. These parameters can help create the baseline code but enables data scientists to actually access code and play around with these parameters easily.

This is AI ML Framework for Code Generation. Use this screen to automate End to end Models lifecycle through a set of processes. By filling in the form, automate following:

- Step 1 - Preprocessing across multiple Sagemaker (includes Standardization, Normalization , Scaling, One-Hot-Encoding etc.).
- Step 2 - Launch and Execute Models through train/tested/validation along with Hyperparameter tuning and best model selection.
- Step 3 - Inference on unseen data before finalizing model for production.
- Step 4 - Automatic packaging of code, artifacts etc. (MRM)

Utilities - EDA

DATA PRE-PROCESSING	MODEL TRAIN/TEST	DATA INFERENCE	PACKAGING/APPROVAL
Container Timeout (sec)	3600		
New Data Path	s3://idf-cl-curved-ue1/hr/employee/attrition/employee-attrition-report		
Identifier Column	NA		
Environment to Deploy	Non-Prod		
Code Generation Framework	Spark		
Algorithm Type	XGBOOST		
Generate Code & Start Experiment			

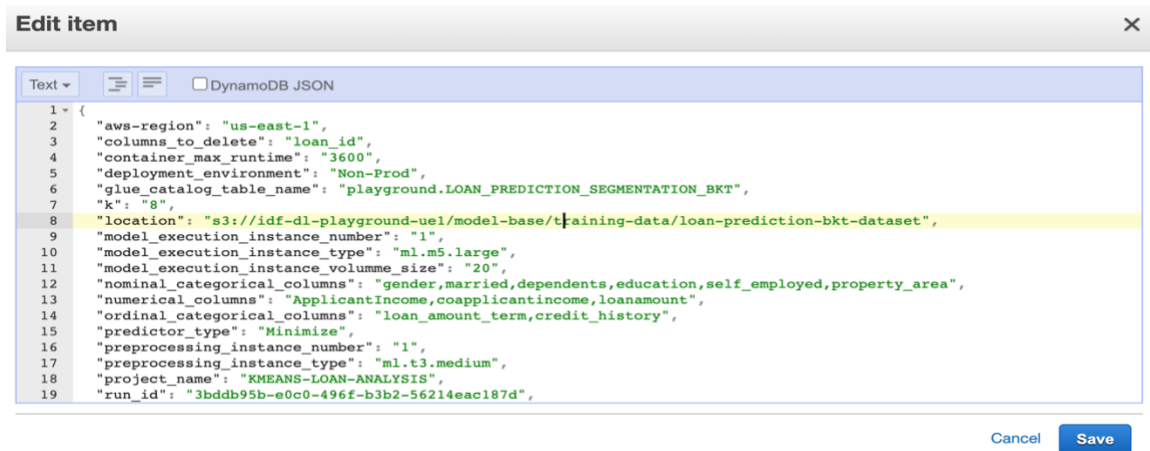
Figure 8 – Train - Test - Validation - Tuning

The above figure provides insight on the repeatability of train-test-validation-tuning process but also enables data scientists to quickly test using *live* data as final step of model evaluation.

Model Repository

A model repository is a key enabler for governing models throughout their lifecycle in this cross-functional environment. IDF provides accelerators to capture and manage the metadata for the end-to-end machine learning lifecycle. The accelerators use a combination of managed AWS services such as Amazon DynamoDB and SageMaker SDKs. The model repository module organizes, tracks, compares, and evaluates the result of the experiments, and performs several crucial functions. The repository also templatises and packages versions of models and experiments along with their technical metadata (e.g., code, libraries) in an inventory for sharing and deployment. Additionally, it captures operational metadata to track every execution of a model, for example, hyperparameters, training datasets, KPIs such performance, and accuracy. IDF capability to capture, track and evaluate model inputs and results in a continuous

way helps customers in validating model performance, drifts and changes in the input, and the output of models. It also enables exploration, searching and discovery of lineage, audit trails, and execution details through APIs.



```

1 {
2   "aws-region": "us-east-1",
3   "columns_to_delete": "loan_id",
4   "container_max_runtime": "3600",
5   "deployment_environment": "Non-Prod",
6   "glue_catalog_table_name": "playground.LOAN_PREDICTION_SEGMENTATION_BKT",
7   "k": "8",
8   "location": "s3://idf-dl-playground-uel/model-base/training-data/loan-prediction-bkt-dataset",
9   "model_execution_instance_number": "1",
10  "model_execution_instance_type": "ml.m5.large",
11  "model_execution_instance_volume_size": "20",
12  "nominal_categorical_columns": "gender,married,dependents,education,self_employed,property_area",
13  "numerical_columns": "ApplicantIncome,coapplicantincome,loanamount",
14  "ordinal_categorical_columns": "loan_amount_term,credit_history",
15  "predictor_type": "Minimize",
16  "preprocessing_instance_number": "1",
17  "preprocessing_instance_type": "ml.t3.medium",
18  "project_name": "KMEANS-LOAN-ANALYSIS",
19  "run_id": "3bddb95b-e0c0-496f-b3b2-56214eac187d",

```

Figure 9 – Model Repository Code

Model Onboarding by Containerization

As companies start moving from on-premises legacy ML solutions to Amazon SageMaker, there is a need to continue supporting legacy models. SageMaker leverages Docker containers to train and serve ML models. The IDF approach is to extend SageMaker's supported algorithms by providing production-grade pre-built container images for common legacy frameworks. Existing models or pipelines can be moved as-is to SageMaker and integrated as part of an end to end inference pipeline, which may include native SageMaker algorithms, and managed through IDF comprehensively.

The following diagram shows how an existing model can be onboarded and managed on AWS serverless training and serving infrastructure.

Feature Repository

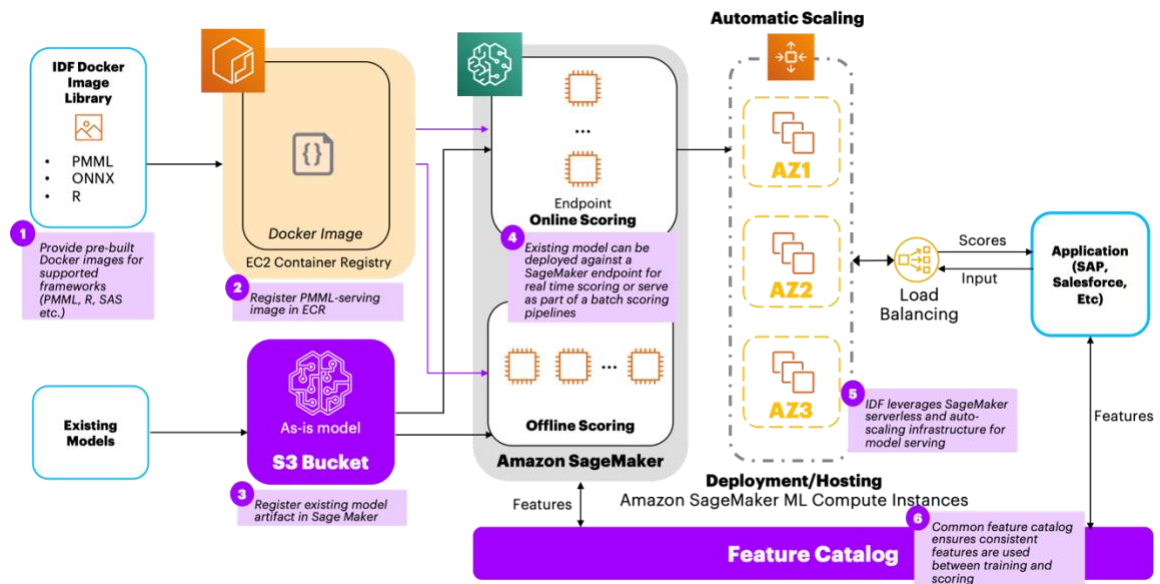


Figure 10 – Existing Model Feature Repository

The features in the repository are observable or measurable entities used in the learning algorithms. A single searchable and editable vault that stores, catalogs, and shares re-usable features of the data, and is an important component for rapidly developing ML models. A feature store provides performant interfaces for reading and writing, and delivers features consistently between training and operational environments that is critical for model performance.

IDF provides accelerators to automate capture and documentation of features during model development and training for consistency. The module uses a combination of managed AWS services such as Amazon DynamoDB to store features.

feature_catalog_table [Close](#)

Overview **Items** Metrics Alarms Capacity Indexes Global Tables [More](#) ▾

[Create item](#) [Actions](#) ▾ [Settings](#) [Refresh](#)

Scan: [Table] feature_catalog_table: run_id ^ Viewing 1 to 4 items

Scan ▾ [Table] feature_catalog_table: run_id ▾

+ Add filter

Start search

☐	run_id ⓘ	aws-region ▾	columns_to_delete
☐	0e25be50-d18f-46ac-a636-db9af48de02f	us-east-1	employeecount,employeenumber,over1
☒	3bddb95b-e0c0-496f-b3b2-56214eac187d	us-east-1	loan_id
☐	72d91a36-e402-4810-a5f9-cacb45288060	us-east-1	duration
☐	e65421e9-f5cc-4a05-81c8-5eada9a265f0	us-east-1	employeecount,employeenumber,over1

Figure 11 – Feature Catalog Table

Explainability Engine

An Explainable AI Engine should have the capability to explain the rationale for AI decisions, characterize the strengths and weaknesses of the decision-making process, and convey an understanding of how they will behave in the future. This capability explores the use of explainable and responsible models (deep explanation, interpretable models and model induction) with an explanation interface (HCI and Psychology features). This in turn opens up the black box decision-making engine. Several ML explainability approaches are in practice today such as SHapley Additive exPlanations (SHAP) and Local Interpretable Model-Agnostic Explanations (LIME). In IDF, the explainability framework can be containerized and integrated as part of an inference pipeline as a post processing step. The post processing step measures the individualized impact of features on a single prediction and store the metrics for further analysis on a visualization tool such as Amazon QuickSight.

The following diagram show how an explainability framework can be integrated as part of an IDF SageMaker inference pipeline.

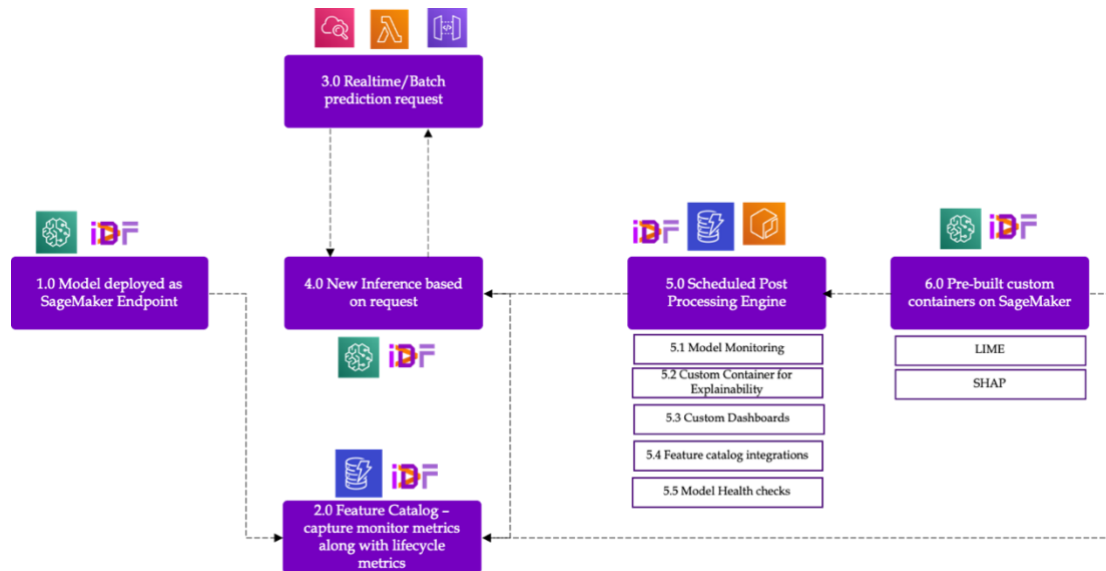


Figure 12 – Explainability Framework

Model Evaluator

Model Evaluation is critical to find the best models that represents the desired outcome. The model evaluator should have the capability to evaluate a model to determine if it will accurately predict the target on new and future data. This is important, if you think about future instances with unknown target values. It is crucial to check the accuracy metric of the ML model on data for which you already know the target answer. This assessment will be used as a proxy for predictive accuracy on future data. Typical approaches used by evaluators include hold-out and cross-validation methods.

Within the IDF Framework, you can understand the importance of executing models using various tuning parameters, objective metric and series of features deduced from the dataset in an automated and repeatable way. The code generator component in IDF generates a model evaluation process using SageMaker Experiments and Model Monitoring SDK's and custom containers. IDF on SageMaker then helps customers evaluate models after every experiment and save the evaluation results in feature catalog repository. Customers can later use these results to identify and package the model selected to be promoted to production. As a by-product of the model evaluation process and capturing the evaluation metrics from both test and validation, we can keep a trend of how each experiment progressed and evaluate the model against an acceptable threshold to help in model acceptance and promotion.

Additionally, this component of the framework explores the use of explainable and responsible models (deep explanation, interpretable models and model induction) with

an explanation interface (HCI and Psychology features). This in turn opens up the black box decision-making engine.

Model Packaging and Deployment Services

A very common scenario for many customers is they have models developed, but deploying the models poses a significant challenge. An approach is needed to make the journey from a reliable and accurate prototype model to a well-performing and scalable production ready application or service. Model packaging and deployment services should provide capabilities to automate the build and deployment of ML models. It doesn't end with automation.

Productionizing ML models requires various practices and disciplines such as statistical modeling, data science, DevOps, ML Engineering, and Data Engineering.

More importantly, recognizing that there are other considerations for a production ready ML application besides the deployment stack, and underscores the need to establish a robust packaging and deployment service. Considerations must be given to continuous improvement and deployment, security, performance, ability to support rapid experimentation, A/B testing, and auto-scaling.

In the IDF Framework, the most important step in the entire MLOps is to help customers seamlessly take a model from data science environment and operationalize the same in production through an automated, repeatable and managed set of processes. As part of the IDF code generator extension and our tight integration with model repository and governance, we help data scientists to come back and select one of the many experiments they think is ready for operationalization. Once they select the right experiment, the code generator framework will automatically download all artifacts and convert them to series of deployment ready code packages using AWS services such as, AWS Step Functions, AWS Lambda, AWS Glue, Amazon ECR, Amazon EKS, and SageMaker. All of the packages are committed using AWS code commit and deployed automatically to production through a one-click Amazon CloudFormation template. This enables us to convert the baseline model to a production ready model through automation and minimal manual intervention.

test.notebook.us-east-1.sagemaker.aws/notebooks/IDF_AIML_Framework_V1.0/Sagemaker_Framework_UI...

jupyter Sagemaker_Framework_UI Last Checkpoint: Last Wednesday at 14:04 (unsaved changes)

File Edit View Insert Cell Kernel Widgets Help Trusted | conda_python3

Code nbdiff

This is AI ML Framework for Code Generation. Use this screen to automate End to end Models lifecycle through a set of processes. By filling in the form, automate following:

Step 1 - Preprocessing across multiple Sagemaker (includes Standardization, Normalization , Scaling, One-Hot-Encoding etc.).

Step 2 - Launch and Execute Models through train/tested/validation along with Hyperparameter tuning and best model selection.

Step 3 - Inference on unseen data before finalizing model for production.

Step 4 - Automatic packaging of code, artifacts etc. (MRM)

Utilities - EDA

DATA PRE-PROCESSING MODEL TRAIN/TEST DATA INFERENCE PACKAGING/APPROVAL

To deploy a model as package in production you would need following:

1. Identify one of the experiments and mark it ready to be deployed

Provide catalog id ready for production

2. Provide commiter's details

Provide commiter's Name

Provide commiter's email

3. We will email/contact Admin with the identified experiment as a package to deploy.

Provide mail id to be used for approval

Figure 13 – Production Ready Packaging of Models

The figure above provides a quick insight of baseline to secondary production ready packaging of models and artifacts through automation and use of AWS native services.

Model Access and Integration

This capability has the objective of democratizing AI by providing wide range access to ML models to consumers and developers through modern ML services such as APIs. Models can be deployed in one of two ways on AWS:

- Batch Transform for batch scoring
- SageMaker Endpoint for online scoring

Both methods leverage SageMaker's cost-effective, scalable and highly available containerized infrastructure.

The model used in the experimentation phase to be ready for production needs to be converted to a production ready codebase that is packaged and ready to be deployed. The code generator framework uses AWS services such as AWS Glue, AWS Lambda, Amazon Step Functions to fetch model artifacts bundle them together to create the

deployment package. The code generator enables you to deploy models with batch or real time inference access.

Model Monitoring Engine

From the beginning, a model deployed to production must be monitored in order to ensure that everything is working properly. When we deploy a model into production, there is an assumption that the data used for predictions have a similar distribution as the trained data sets. It is expected that data distributions will drift over time, so deploying a model into production is not a one-time process but a continuous process as model performance degrades over time.

One approach to mitigate the effects of drift on model performance is to re-train the model periodically or on an ad-hoc basis. This strategy is typically adopted by companies that are very early in their ML journey. However, as companies scale their ML operations, a more sophisticated approach is needed but would require a very strong ML foundation, monitoring capabilities, process, automation, and human oversight.

A better approach is to continuously monitor the incoming data and re-train your model on newer data if the data distribution has deviated significantly from the original training data distribution. There are a number of statistical metrics (mean, standard deviations, min, max, and etc.) that can be captured per feature. Amazon SageMaker Model monitor provides the capability to continuously monitor ML models in Production. Model monitor uses rules to infer data drifts on real-life data and can automatically trigger the re-training pipeline based on acceptability thresholds.

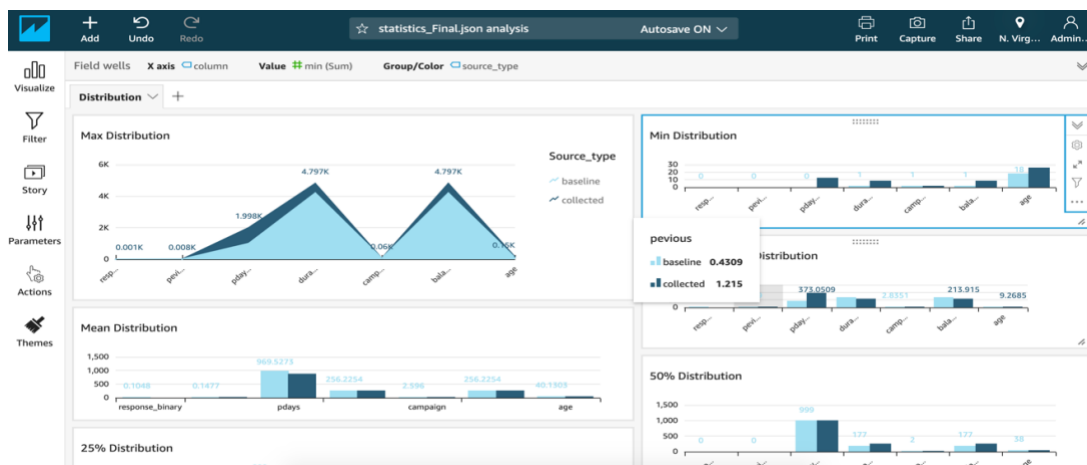


Figure 14 – Amazon SageMaker Model Monitor Dashboard

A re-training process runs in the automation environment on the new observed data set and is a simplified version of the training pipeline. The features, pre-processing step, hyperparameters selection remain the same. However, if the objective metric, e.g., accuracy, of the re-training job is not met, a human review and approval process is required and a model review and tuning exercise is expected in an experimentation environment. A re-trained model artifact can be deployed directly in production and can leverage Sagemaker's production variants if A/B testing is required

The following figure shows how a serverless workflow can be built using a combination of AWS services and ML Ops accelerators to automate model re-training and deployment.

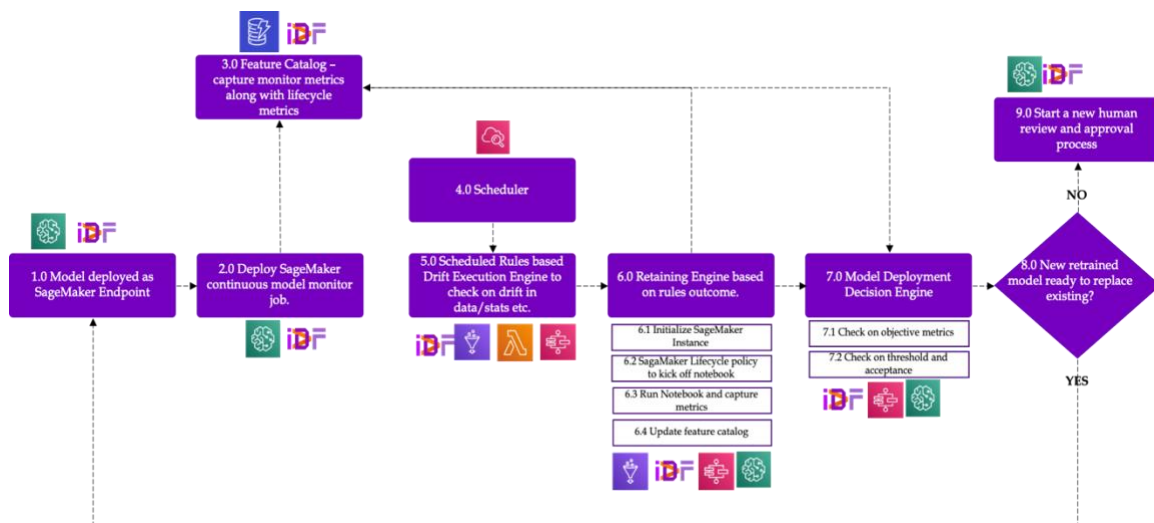


Figure 15 – Serverless Workflow

Model Lifecycle Governance

Reproducibility, traceability, and verifiability are enabled by several simple capabilities provided by the ML model lifecycle governance. These capabilities include capturing artifacts about models as they evolve over the data science lifecycle, storing these artifacts, and, searching and viewing artifacts. These capabilities are enabled with slight additions to the current data science lifecycle. Model lifecycle governance enables engineers and data scientists to visualize, categorize, group, manage, and govern ML models. Architecturally, IDF augments SageMaker with the following capabilities for effective model lifecycle governance:

- A catalog or a database that lists models, including when they were tested, trained, and deployed.

- A catalog of validation data sets and the accuracy measurements of stored models.
- Versioning (of models, feature vectors, data) and the ability to roll out, roll back, or have multiple live versions.
- Metadata and artifacts needed for a full audit trail.
- Who approved and pushed the model out to production, who is able to monitor its performance and receive alerts, and who is responsible for the model?

Conclusion

Great success in machine learning has led to a torrent of artificial intelligence (AI) applications. Continued advances promise to produce autonomous systems that will perceive, learn, decide, and act on their own. However, the effectiveness of these systems is limited by the challenge to effectively industrialize these applications at scale and a machine's current inability to explain their decisions and actions to human users. It is clear that there is a gap between model development and effective industrialization. To mitigate these challenges, Machine Learning Engineering requires a shift in perspective and a focus on enhancing collaboration, developing standards, and deployment at scale.

New machine learning engineering systems will have the ability to explain their rationale, characterize their strengths and weaknesses, and convey an understanding of how they will behave in the future. Machine Learning engineering gathers every process that enables AI algorithms to be implemented by enabling the enterprise to Scale ML, achieve model deployment while providing data engineering pipeline automation, responsible and explainable AI for the company. This paper provides an overview for companies to use as a starting point and understand this evolving landscape and familiarize themselves with appropriate architectural options.

Call to action

To begin building your companies new ML Engineering solution, consider the following:

- **Data Inventory:** Start with your most frequently accessed and time-relevant data. This will your initial sources to establish a version of truth.
- **Data Hubs:** Identify current data repositories to understand current needs across the business and the extent to which these data hubs meet the business requirements.

- **Use Cases:** Examine the business use cases and align these use cases to existing solutions. Identify potential opportunities.
- **Prioritize individual AI/ML use cases.** Prioritizing helps you develop a roadmap for implementing the new ML Engineering at scale
- **Models and AI/ML:** Identify the right contextual and analytical models to cover the business use cases.
- **Establish** the right tooling and integrate industrialization frameworks for data scientists and engineers.
- **Define** a clear path to production for your models and AI/ML applications.
- **Select** the data technology stack for your data and research the optimal deployment methods.
- **Promoting** resources that best prepare aspiring professionals.

Contributors

Contributors to this document include:

- Karthik Sonti, Principal Solutions Architect, AI/ML, Amazon Webservices
- Atish Ray, Managing Director, Accenture Applied Intelligence
- Denis Leung Pah Hang, Sr. Manager, Applied Intelligence, Accenture
- Nayanjyoti Paul, Manager, Applied Intelligence, Accenture

Document Revisions

Date	Description
October 2020	First publication

Notes

¹ <https://www.accenture.com/us-en/insights/artificial-intelligence/ai-investments>

² https://www.amazon.com/Machine-Learning-Engineering-Andriy-Burkov/dp/1999579577/ref=sr_1_5?dchild=1&keywords=machine+learning+engineering&qid=1600717940&sr=8-5

³ https://www.accenture.com/us-en/insights/technology/_acnmedia/Thought-Leadership-Assets/PDF-2/Accenture-Technology-Vision-2020-Full-Report.pdf

⁴ <https://www.accenture.com/us-en/insights/artificial-intelligence/scaling-enterprise-ai>

⁵ <https://www.accenture.com/us-en/insights/technology/technology-trends-2020>

⁶ <https://www.technologyreview.com/2019/01/21/137783/algorithms-criminal-justice-ai/#:~:text=Using%20historical%20data%20to%20train,the%20mistakes%20of%20the%20past>

About Accenture

Accenture is a leading global professional services company, providing a broad range of services in strategy and consulting, interactive, technology and operations, with digital capabilities across all of these services. We combine unmatched experience and specialized capabilities across more than 40 industries — powered by the world's largest network of Advanced Technology and Intelligent Operations centers. With 506,000 people serving clients in more than 120 countries, Accenture brings continuous innovation to help clients improve their performance and create lasting value across their enterprises. Visit us at www.accenture.com.

Accenture does not provide legal, regulatory, audit, or tax advice. Readers are responsible for obtaining such advice from their own legal counsel or other licensed professionals.